

Lab 9: Autonomous Target Tracking via Computer Vision

Student Name:	Anas Gulzar
Registration Number:	2022-MC-07
Institution:	University of Engineering and Technology (UET) Lahore
Course Title:	MCT-454L Mobile Robotics
Environment:	Gazebo (TurtleBot3 Waffle)

1. Primary Goals

The main purpose of this experiment was to build an automated visual tracking loop. By utilizing the camera attached to the simulated TurtleBot3, we extracted live image frames, isolated specific colors using OpenCV, found their central coordinates, and commanded the robot to autonomously align with and drive toward those objects.

2. Simulation Preparation

We initialized the Gazebo environment with the Waffle variant of the robot. To ensure the camera was working, we inspected the video output using the `rqt_image_view` tool. To test our tracking code, we generated custom stationary obstacles—specifically, a red cylinder and a blue box—by feeding SDF code directly into the terminal using `spawn_entity.py`.

3. Codebase Organization

All scripts were bundled into a new Python-based ROS 2 package named `lab9_camera_cv_utils`. The logic was split across five distinct files, each handling a specific milestone of the overall system (from basic image viewing to full autonomous pursuit).

4. Step-by-Step Execution

Phase 1: Accessing the Video Stream

We wrote a listener node to pull data from `/camera/image_raw`. Using `cv_bridge`, the ROS data was translated into standard OpenCV images. To keep the interface manageable, the output window was scaled down by 50%.

Phase 2: Filtering by Color (HSV)

Because lighting conditions fluctuate, we converted the image format from BGR to HSV. After slightly blurring the image to cut down on visual static, we applied strict numerical limits to isolate specific colors (Red, Green, Grey, Black). Since the color red splits across the beginning and end of the Hue spectrum, we had to stitch two separate ranges together. We also used morphological filters (Erosion and Dilation) to erase background noise and fill in hollow spots within the detected shapes.

Phase 3: Locating the Object's Center

Once the colors were isolated into binary masks, we used contour mapping to draw outlines around the shapes. By ignoring tiny blobs (under 500 pixels), we isolated the primary target. We then applied image moments to calculate the exact X and Y coordinates (the centroid) of the largest object, marking it with a dot on the screen.

Phase 4: Steering with Math (Proportional Control)

To make the robot face the target, we calculated the difference between the object's X-coordinate and the exact center of the camera view. This error value was multiplied by a small gain factor ($k_p = 0.0005$) to generate an angular velocity command. A small 3-pixel "dead zone" was added so the robot wouldn't jitter when it was perfectly lined up. If no object was visible, the robot was instructed to spin in place at 0.3 rad/s to scan the area.

Phase 5: Full Chase and Halt Sequence

The final script merged all previous steps into a smart state machine. The robot scans for colors based on a hierarchy (Red first, then Blue). Once it locks on and centers the object, it drives straight at 0.15 m/s. Instead of hardcoding a stopping distance, the robot halts when the colored object takes up 65% of its screen. If the target vanishes abruptly, it performs a full 360-degree sweep to reacquire it before completely shutting down.

5. Technical Insights and Tuning

- **HSV Superiority:** The HSV color model was far more resilient to simulation shadows than BGR.
- **Managing False Positives:** The floor in the simulation often tricked the grey/black filters. Increasing the saturation requirements helped filter out these dull background colors.
- **Controller Balance:** Finding the right k_p was crucial. Too high (e.g., 0.005) caused severe oversteering, while too low made the robot painfully slow to react.
- **Dynamic Stopping:** Relying on the percentage of the screen filled by the object proved to be a highly reliable and resolution-independent way to prevent collisions.

6. Summary of Achievements

This laboratory session successfully verified that a robot can navigate its environment relying purely on visual data, without needing laser scanners or wheel encoders. We bridged the gap between raw pixel data, feature extraction, and mechanical control, resulting in a fully functional, self-correcting chase sequence.